Modelado matemático y optimización cuadrática del portafolio de inversión

1. Modelo de Markowitz

El modelo de Markowitz es una herramienta de optimización financiera que permite determinar la distribución del capital entre distintos instrumentos financieros, considerando la relación entre el rendimiento esperado y el riesgo de la inversión. En el presente proyecto, se utiliza el modelo de portafolio de mínima varianza global (*Global Minimum Variance Portfolio*, GMVP), cuyo objetivo es encontrar la combinación de activos que minimice el riesgo total del portafolio.

El modelo considera cuatro instrumentos financieros: DUK, LQD, KIMBERA.MX y AC.MX. Para su implementación se utilizan el vector de rendimientos esperados y la matriz de varianzas y covarianzas estimados a partir de los datos históricos.

2. Variables de decisión

Se define el vector de pesos del portafolio como:

$$\boldsymbol{w} = (w_1 , w_2 , w_3 , w_4)^T$$

Donde cada variable representa la proporción del capital invertida en un instrumento:

- w_1 : proporción invertida en DUK.
- w_2 : proporción invertida en LQD.
- w_3 : proporción invertida en KIMBERA.MX.
- w_4 : proporción invertida en AC.MX.

Los pesos constituyen las variables de decisión del problema, ya que son los valores que el algoritmo debe determinar para obtener el portafolio óptimo.

3. Función objetivo

La función objetivo consiste en minimizar la varianza del portafolio:

$$\min_{w} \quad \frac{1}{2} w^T \sum w$$

La matriz $\boldsymbol{Sigma}$ representa la matriz de varianzas y covarianzas de los rendimientos de los activos. La varianza del portafolio se expresa como:

$$\sigma_p^2 = w^T \sum w$$

Esta expresión considera tanto el riesgo individual de cada instrumento como las relaciones de covarianza entre ellos. Por esta razón, la diversificación puede reducir el riesgo total del portafolio, dependiendo de cómo se comporten conjuntamente los rendimientos de los activos.

El factor (1/2) se incorpora por conveniencia matemática y no modifica la solución óptima, debido a que multiplicar la función objetivo por una constante positiva no altera el punto que la minimiza.

4. Restricciones del modelo

La optimización se encuentra sujeta a dos restricciones principales.

Restricción presupuestaria:

$$1^T w = 1$$

Equivalentemente:

$$w_1 + w_2 + w_3 + w_4 = 1$$

Esta condición establece que el capital debe distribuirse completamente entre los cuatro instrumentos, de modo que los pesos sumen el 100 % de la inversión.

Restricción de no negatividad:

$$w_i \geq 0, \qquad i = 1, 2, 3, 4$$

Esta restricción impide asignar pesos negativos, por lo que no se permiten posiciones cortas. Bajo la restricción presupuestaria, también implica que ningún instrumento puede representar más del 100 % del capital.

En consecuencia, el problema completo queda formulado como:

$$\min_{w} \quad \frac{1}{2} w^T \sum w$$

Sujeto a:

$$w_1 + w_2 + w_3 + w_4 = 1$$

$$w_1 , w_2 , w_3 , w_4 \geq 0$$

5. Naturaleza cuadrática y convexidad

La función objetivo es cuadrática porque contiene productos entre los pesos de inversión y los elementos de la matriz de covarianzas. Las restricciones, por su parte, son lineales.

La matriz de covarianzas es simétrica y semidefinida positiva, propiedad que garantiza la convexidad de la función objetivo. Si la matriz es definida positiva, la función es estrictamente convexa y, bajo las restricciones convexas del modelo, existe una única solución óptima global.

En los datos empleados en este proyecto, los cuatro valores propios de la matriz de covarianzas son positivos. Por lo tanto, la matriz es definida positiva y la función objetivo es estrictamente convexa. Esto permite concluir que el problema tiene una solución global única.

6. Algoritmo de resolución: SLSQP

Para resolver el problema se utiliza la función `minimize` de la biblioteca `scipy.optimize` de Python, mediante el método SLSQP (*Sequential Least Squares Programming*).

Este algoritmo permite resolver problemas de optimización con restricciones de igualdad y desigualdad. En este caso, busca los pesos que minimizan la varianza del portafolio, respetando la condición de que la suma de los pesos sea igual a uno y que ninguno sea negativo.

El procedimiento consiste en definir la función objetivo, proporcionar una estimación inicial de los pesos, establecer las restricciones y ejecutar el algoritmo. Una vez concluida la optimización, se verifican la convergencia del método y el cumplimiento de las restricciones.

7. Supuestos y limitaciones

El modelo considera los siguientes supuestos:

1. Los rendimientos esperados y la matriz de covarianzas se estiman a partir de información disponible y se consideran parámetros conocidos durante la optimización.
2. Todos los instrumentos se evalúan sobre un horizonte temporal y una frecuencia de rendimiento consistentes.
3. El capital se distribuye completamente entre los activos considerados.
4. No se permiten posiciones cortas ni pesos negativos.
5. El riesgo del portafolio se mide mediante la varianza de sus rendimientos.
6. Se omiten costos de transacción, impuestos y otras fricciones de mercado, salvo que se incorporen expresamente.
7. La solución óptima depende de las estimaciones utilizadas, por lo que no garantiza rendimientos futuros ni elimina el riesgo de inversión.

8. Resultado esperado

El resultado del modelo es un vector de pesos óptimos que indica qué porcentaje del capital debe asignarse a cada instrumento para minimizar la varianza del portafolio bajo las restricciones establecidas. Los pesos pueden expresarse como porcentajes multiplicando cada valor por cien.

La solución se acompaña del rendimiento esperado y de la volatilidad estimada del portafolio, lo que permite interpretar el resultado y evaluar sus características de riesgo y rendimiento.

9. Conclusión

A partir de la implementación del modelo de Markowitz y del algoritmo de optimización SLSQP en Python, se obtuvo una distribución del capital orientada a minimizar la varianza del portafolio integrado por DUK, LQD, KIMBERA.MX y AC.MX. Los resultados indican que la asignación óptima corresponde aproximadamente a **6.72 % en DUK, 86.46 % en LQD, 2.83 % en KIMBERA.MX y 3.99 % en AC.MX**, con lo que se distribuye el 100 % del capital disponible.

Esta distribución muestra que el modelo asigna la mayor proporción de la inversión a LQD, mientras que los porcentajes restantes se distribuyen entre los otros tres instrumentos. Esto se debe a que el algoritmo busca la combinación de pesos que genera la menor varianza posible, considerando las varianzas individuales y las covarianzas entre los activos. Por lo tanto, los porcentajes obtenidos representan la solución matemática del problema de mínima varianza bajo las restricciones establecidas, y no una selección basada exclusivamente en cuál activo presenta el mayor rendimiento esperado.

Con esta distribución, el portafolio presenta un rendimiento esperado aproximado de **5.89 %**, una varianza de **0.0050003** y una volatilidad cercana al **7.07 %**, de acuerdo con los datos utilizados en el modelo. Estos indicadores permiten evaluar las características estimadas de la cartera y sirven como referencia para analizar si la distribución propuesta se ajusta a los objetivos y al nivel de riesgo que se está dispuesto a asumir. El rendimiento y la volatilidad corresponden a la misma periodicidad de los datos de entrada, por lo que deben interpretarse de acuerdo con la frecuencia con la que se calcularon los rendimientos históricos.

En términos prácticos, los porcentajes obtenidos pueden utilizarse como una propuesta de distribución inicial del capital. Por ejemplo, por cada $10,000 disponibles para invertir, la asignación sería de aproximadamente $672 en DUK, $8,646 en LQD, $283 en KIMBERA.MX y $399 en AC.MX. Esta distribución permite trasladar los resultados matemáticos a cantidades monetarias concretas, facilitando la interpretación y aplicación del modelo.

Finalmente, se concluye que la optimización cuadrática constituye una herramienta útil para apoyar la toma de decisiones financieras, ya que permite establecer una distribución del capital con base en una medida cuantitativa del riesgo. Sin embargo, la elevada concentración en LQD también evidencia una limitación importante: minimizar la varianza no garantiza una diversificación equilibrada ni el mayor rendimiento posible. Por ello, antes de utilizar estos porcentajes en una inversión real, sería conveniente revisar la calidad y periodicidad de los datos, considerar los costos de transacción y evaluar si la concentración obtenida es compatible con los objetivos del inversionista. Asimismo, los resultados dependen de las estimaciones históricas y no garantizan que el comportamiento futuro de los activos sea el mismo.

Explicación del código de optimización del portafolio de inversión mediante el modelo de Markowitz

1. Importación de librerías

El código comienza con la importación de dos librerías de Python: `NumPy` y `SciPy`.

```
import numpy as np
from scipy.optimize import minimize
```

La librería `NumPy` permite realizar operaciones matemáticas con arreglos y matrices, como multiplicaciones matriciales, sumas y cálculos de varianzas. Por su parte, la función `minimize`, perteneciente al módulo `scipy.optimize`, permite encontrar los valores que minimizan una función matemática, considerando las restricciones establecidas en el modelo.

En este caso, ambas herramientas se utilizan para calcular la distribución óptima del capital entre los activos seleccionados, buscando minimizar el riesgo del portafolio.

2. Definición de los activos financieros

```
activos = ["DUK", "LQD", "KIMBERA.MX", "AC.MX"]
```

En esta sección se define una lista con los cuatro activos financieros que integran el portafolio: DUK, LQD, KIMBERA.MX y AC.MX.

Cada activo representa una alternativa de inversión a la que se le asignará una proporción del capital disponible. El objetivo del modelo es determinar qué porcentaje debe invertirse en cada instrumento para obtener el portafolio con la menor varianza posible, respetando las restricciones establecidas.

3. Definición de los rendimientos esperados

```
mu = np.array([
    0.12795459959881655,
    0.048374773852154065,
    0.08395689346754248,
    0.1517123938055161
])
```

El vector `mu` contiene los rendimientos esperados de los cuatro activos financieros. Cada valor corresponde al rendimiento estimado de un instrumento, expresado en forma decimal.

Por ejemplo, un rendimiento de `0.12795` equivale aproximadamente a un 12.80 %. Estos valores permiten calcular el rendimiento esperado total del portafolio una vez determinados los pesos óptimos de inversión.

Es importante señalar que, en la formulación utilizada, los rendimientos esperados no intervienen directamente en la función objetivo, debido a que el modelo busca minimizar exclusivamente la varianza del portafolio. Sin embargo, se utilizan posteriormente para calcular el rendimiento esperado de la combinación de activos seleccionada.

4. Definición de la matriz de varianzas y covarianzas

```
Sigma = np.array([
    [0.02836283160471076, 0.0033342848638640388,
     0.002924452969247777, 0.0032471777671372742],
    [0.0033342848638640388, 0.005329421199429245,
     0.00276230671326149, 0.002260777429451611],
    [0.002924452969247777, 0.00276230671326149,
     0.059733581620689376, 0.018137591232738736],
    [0.0032471777671372742, 0.002260777429451611,
     0.018137591232738736, 0.05798654155415723]
])
```

La variable `Sigma` representa la matriz de varianzas y covarianzas de los rendimientos de los activos. Esta matriz es fundamental para el modelo de Markowitz, ya que permite medir el riesgo individual de cada instrumento y la relación entre los rendimientos de los diferentes activos.

Los elementos de la diagonal principal representan las varianzas individuales de los activos. En cambio, los elementos que se encuentran fuera de la diagonal representan las covarianzas entre pares de instrumentos, es decir, indican cómo tienden a variar sus rendimientos conjuntamente.

La matriz es simétrica, lo que significa que la covarianza entre dos activos es la misma independientemente del orden en que se consideren. Esta información permite evaluar el efecto de la diversificación, ya que el riesgo total no depende únicamente del riesgo individual de cada activo, sino también de la relación que existe entre ellos.

Matemáticamente, la varianza del portafolio se expresa como:

$$\sigma_p^2 = w^T \sum w$$

Donde:

- σ_p^2 representa la varianza del portafolio.
- w es el vector de pesos de inversión.
- Σ es la matriz de varianzas y covarianzas.
- w^T representa el vector de pesos transpuesto.

5. Definición de la función objetivo

```
def funcion_objetivo(w):
    return 0.5 * w @ Sigma @ w
```

La función `funcion_objetivo` establece el criterio matemático que el algoritmo debe minimizar. En este caso, se utiliza la varianza del portafolio, representada mediante la expresión matricial $w^T \sum w$.

El símbolo `@` de Python representa la multiplicación matricial. Por lo tanto, la instrucción `w @ Sigma @ w` calcula la varianza del portafolio a partir de los pesos de inversión y la matriz de varianzas y covarianzas.

El factor `0.5` se incorpora para simplificar la expresión del gradiente de la función objetivo. Como se trata de una constante positiva, no modifica la combinación de pesos que minimiza la función.

La función objetivo corresponde a un problema de optimización de mínima varianza global (*Global Minimum Variance Portfolio*), cuyo propósito es encontrar la combinación de activos que presenta el menor riesgo posible, de acuerdo con la matriz de covarianzas utilizada.

Su formulación matemática es:

$$\min_w \quad \frac{1}{2} w^T \sum w$$

6. Cálculo del gradiente

```
def gradiente(w):
    return Sigma @ w
```

El gradiente es un vector que contiene las derivadas parciales de la función objetivo con respecto a cada peso del portafolio. En términos prácticos, proporciona información sobre cómo cambia la función objetivo cuando se modifica la proporción invertida en cada activo.

En el código se calcula mediante la multiplicación de la matriz `Sigma` por el vector de pesos `w`. Esta expresión corresponde al gradiente de la función objetivo definida anteriormente, que incluye el factor de un medio.

El gradiente se proporciona directamente al algoritmo de optimización para facilitar el proceso de búsqueda de la solución y mejorar su eficiencia numérica.

7. Establecimiento de las restricciones

```
restricciones = [
    {
        "type": "eq",
        "fun": lambda w: np.sum(w) - 1,
        "jac": lambda w: np.ones_like(w)
    }
]
```

En esta sección se establece la restricción que obliga a que la suma de los pesos del portafolio sea igual a uno. Esto significa que el 100 % del capital disponible debe distribuirse entre los cuatro activos.

La instrucción `np.sum(w) - 1` expresa matemáticamente la restricción como una igualdad que debe ser igual a cero. Por ejemplo, si los pesos suman exactamente uno, la restricción se cumple.

Por su parte, `type: "eq"` indica que se trata de una restricción de igualdad. La expresión `np.ones_like(w)` proporciona el gradiente de esta restricción, formado por unos, ya que la derivada de la suma de los pesos con respecto a cada uno de ellos es igual a uno.

La restricción se representa matemáticamente como:

$$1^T w = 1$$

Donde 1 es un vector formado por unos y w es el vector de pesos de inversión.

8. Establecimiento de los límites de inversión

```
limites = [(0, 1) for _ in activos]
```

Los límites determinan los valores mínimos y máximos que puede tomar cada peso del portafolio. En este caso, cada activo tiene un límite inferior de cero y un límite superior de uno.

Esto significa que ningún activo puede tener un peso negativo y que la inversión en cada instrumento no puede superar el 100 % del capital disponible. Por lo tanto, el modelo permite únicamente posiciones largas y excluye las ventas en corto.

La restricción se expresa de la siguiente manera:

$$0 \leq w_i \leq 1, \qquad i = 1, 2, 3, 4$$

Donde w_i representa el peso asignado a cada uno de los cuatro activos.

9. Definición de los pesos iniciales

```
w_inicial = np.ones(len(activos)) / len(activos)
```

Antes de iniciar la optimización, el algoritmo necesita una solución inicial a partir de la cual comenzar la búsqueda. En este caso, se establece una distribución equitativa del capital entre los cuatro activos.

Como el portafolio está integrado por cuatro instrumentos, cada uno recibe inicialmente un peso de 0.25, equivalente al 25 % de la inversión.

Estos pesos iniciales no representan necesariamente la solución óptima. Su función consiste en proporcionar un punto de partida para que el algoritmo encuentre la combinación que minimiza la varianza del portafolio.

10. Ejecución del algoritmo de optimización

```
resultado = minimize(
    fun=funcion_objetivo,
    x0=w_inicial,
    jac=gradiente,
    method="SLSQP",
    bounds=limites,
    constraints=restricciones,
    options={"maxiter": 1000, "ftol": 1e-12, "disp": False}
)
```

En esta sección se ejecuta el proceso de optimización mediante la función `minimize`. Se proporcionan la función objetivo, los pesos iniciales, el gradiente, los límites de inversión y la restricción de igualdad.

El método seleccionado es SLSQP (*Sequential Least Squares Programming*), un algoritmo que permite resolver problemas de optimización con restricciones de igualdad y desigualdad, así como límites sobre las variables.

Los parámetros utilizados tienen las siguientes funciones:

- `fun=funcion_objetivo`: indica la función que debe minimizarse.
- `x0=w_inicial`: establece los pesos iniciales del portafolio.
- `jac=gradiente`: proporciona el gradiente de la función objetivo.
- `method="SLSQP"`: selecciona el algoritmo de optimización.
- `bounds=limites`: establece los límites mínimo y máximo de cada peso.
- `constraints=restricciones`: incorpora la condición de que los pesos sumen uno.
- `maxiter=1000`: permite realizar hasta 1,000 iteraciones durante la búsqueda de la solución.
- `ftol=1e-12`: establece una tolerancia numérica estricta para el criterio de convergencia.
- `disp=False`: evita que el algoritmo muestre mensajes detallados durante su ejecución.

El resultado se almacena en la variable `resultado`, que contiene información sobre los pesos obtenidos, el estado de la optimización y otros datos relacionados con la solución.

11. Verificación de la convergencia y cálculo de resultados

```
if resultado.success:
    pesos = resultado.x
    rendimiento = mu @ pesos
    varianza = pesos @ Sigma @ pesos
    volatilidad = np.sqrt(varianza)
else:
    print("La optimizacion no convergio.")
    print("Mensaje:", resultado.message)
```

Una vez finalizado el proceso de optimización, el programa verifica si el algoritmo encontró una solución satisfactoria mediante la condición `resultado.success`.

Si la optimización fue exitosa, los pesos óptimos se almacenan en la variable `pesos`. Cada valor representa la proporción del capital que debe asignarse al activo correspondiente, de acuerdo con el modelo de mínima varianza.

Posteriormente, se calculan tres indicadores principales:

a) Rendimiento esperado del portafolio

La instrucción `rendimiento = mu @ pesos` calcula el rendimiento esperado del portafolio mediante la suma ponderada de los rendimientos esperados de los activos.

Matemáticamente, se expresa como:

$$E(R_p) = \mu^T w$$

Donde $E(R_p)$ es el rendimiento esperado del portafolio, μ es el vector de rendimientos esperados y w es el vector de pesos óptimos.

b) Varianza del portafolio

La instrucción `varianza = pesos @ Sigma @ pesos` calcula la varianza del portafolio utilizando los pesos óptimos y la matriz de varianzas y covarianzas.

Este indicador representa una medida de la dispersión de los rendimientos y se utiliza como criterio principal para evaluar el riesgo en el modelo de Markowitz.

c) Volatilidad del portafolio

La instrucción `volatilidad = np.sqrt(varianza)` calcula la raíz cuadrada de la varianza. El resultado corresponde a la desviación estándar de los rendimientos del portafolio, conocida también como volatilidad.

A diferencia de la varianza, la volatilidad se expresa en las mismas unidades que los rendimientos. Por ejemplo, si se trabaja con rendimientos decimales, una volatilidad de 0.10 equivale al 10 %.

En caso de que la optimización no sea exitosa, el programa muestra un mensaje indicando que no se alcanzó la convergencia y presenta la información proporcionada por el algoritmo mediante `resultado.message`. Esto permite identificar posibles problemas durante el proceso de cálculo.

12. Interpretación general del modelo

En conjunto, el código implementa un modelo de optimización cuadrática basado en la teoría de portafolios de Harry Markowitz. Su finalidad es determinar los pesos de inversión que minimizan la varianza total de una cartera compuesta por cuatro activos financieros.

El modelo considera tanto las varianzas individuales como las covarianzas entre los instrumentos. De esta manera, la distribución óptima no depende exclusivamente del riesgo de cada activo por separado, sino también de la forma en que sus rendimientos se relacionan entre sí.

Las restricciones garantizan que la totalidad del capital se distribuya entre los activos y que no se asignen pesos negativos. Una vez obtenida la solución, se calculan el rendimiento esperado, la varianza y la volatilidad del portafolio para facilitar su interpretación financiera.

Es importante destacar que el resultado corresponde al portafolio de mínima varianza dentro de las condiciones establecidas, no necesariamente al portafolio con el mayor rendimiento esperado. Además, la calidad de los resultados depende de la estimación de los rendimientos y de la matriz de varianzas y covarianzas utilizada como entrada del modelo.